

## ✅ UPDATED DAY 11 — PDF TABLES (CORRECT ARCHITECTURE)

Now we support:

- ✅ Excel → separate context
- ✅ PDF Tables → separate context
- ✅ RAG → separate context

WITHOUT merging unrelated datasets.

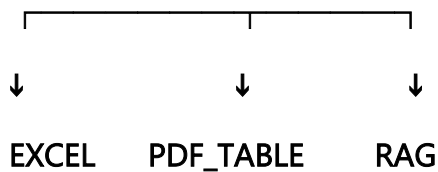
---

## ✅ FINAL FLOW

User Query



Planner



## ✅ STEP 1 — CREATE PDF TABLE EXTRACTOR

📌 Create: ingestion/pdf\_table\_extractor.py

---

## ✅ Code

```
import camelot
```

```
def extract_pdf_tables(pdf_path):
```

```
    tables = camelot.read_pdf(
```

```
        pdf_path,
```

```
        pages="all"
```

```
    )
```

```
    dfs = []
```

for table in tables:

```
df = table.df
```

```
# ✅ use first row as columns
```

```
df.columns = df.iloc[0]
```

```
df = df[1:]
```

```
# ✅ clean column names
```

```
df.columns = [
```

```
    str(c).strip().lower()
```

```
    for c in df.columns
```

```
]
```

```
dfs.append(df)
```

```
return dfs
```

---

✅ ✅ STEP 2 — UPDATE main.py

📌 app/main.py

---

✅ Add imports

```
import pandas as pd
```

```
from tools.excel_agent import ExcelAgent
```

```
from ingestion.pdf_table_extractor import extract_pdf_tables
```

---

✅ Add PDF table loading

AFTER:

```
docs = load_pdf(r"data/raw/research_test_file.pdf")
```

```
vs.index(docs)
```

ADD:

```
# ✅ PDF TABLE EXTRACTION
```

```
pdf_tables = extract_pdf_tables(  
    r"data/raw/research_test_file.pdf"  
)  
pdf_table_agent = None  
if pdf_tables:  
    combined_pdf_df = pd.concat(  
        pdf_tables,  
        ignore_index=True  
    )  
    pdf_table_agent = ExcelAgent(  
        combined_pdf_df  
    )  
    print("PDF Table Agent Ready ✅")
```

---

✅ Keep your existing Excel agent

```
excel_agent = ExcelAgent(df)
```

NO CHANGES.

---

✅ Replace agent creation

❌ OLD: agent = Agent(vs, excel\_agent)

---

✅ NEW:

```
agent = Agent(  
    rag_system=vs,  
    excel_tool=excel_agent,  
    pdf_table_tool=pdf_table_agent )
```

---

### ✓ ✓ STEP 3 — UPDATE agent.py

---

#### ✓ Update init

class Agent:

```
def init(
    self,
    rag_system,
    excel_tool=None,
    pdf_table_tool=None
):
    self.rag = rag_system
    self.excel = excel_tool
    self.pdf_table = pdf_table_tool

    self.planner = Planner()
    self.memory = ChatMemory()
```

---

### ✓ ✓ STEP 4 — UPDATE planner.py

Modify prompt:

```
prompt = f"""
```

Break the query into steps.

Tools:

- EXCEL → spreadsheet dataset
- PDF\_TABLE → tables extracted from PDFs
- RAG → document text retrieval

Examples:

Q: average car price

Step 1: EXCEL: average car price

Q: average employee salary from pdf

Step 1: PDF\_TABLE: average employee salary

Q: summarize report

Step 1: RAG: summarize report

Now:

Q: {query}

""""

---

## ✅✅ STEP 5 — UPDATE run() EXECUTION

Inside execution loop:

if tool == "EXCEL" and self.excel:

    result = self.excel.smart\_query(task, generate)

elif tool == "PDF\_TABLE" and self.pdf\_table:

    result = self.pdf\_table.smart\_query( task, generate)

elif tool == "RAG":

    docs = self.rag.search(task)

    result = "\n".join(docs[:3])

else:

    result = "Unknown tool"

---

## ✅✅ NOW YOUR SYSTEM SUPPORTS

---

### ✅ Example 1

Excel → Cars dataset

PDF → Employee salary table

Query: average car price

✅ Uses: EXCEL

---

✅ Example 2

Query: average employee salary from pdf

✅ Uses: PDF\_TABLE

---

✅ Example 3

Query: summarize research paper

✅ Uses: RAG

---

✅ THIS IS THE CORRECT DESIGN

Because: Different semantic domains

→ separate agents/tools ✅

instead of: merge everything into one dataframe ❌

---

✅ YOUR SYSTEM NOW

✓ RAG ✅

✓ Excel Agent ✅

✓ PDF Table Agent ✅

✓ Planner ✅

✓ Memory ✅

✓ Multi-source reasoning ✅ 🔥

---

🚀 NEXT (Day 12)

Self-correcting agent

(retry when execution fails)